

UART Receiver

MIT Fall 2025

The questions below are due on Wednesday September 24, 2025; 11:59:00 PM.

Now that you've written a transmitter for UART, it's time to cover the other side and build a receiver. Unlike the SPI receiver from last week, we don't have a clock to determine when to read our data, since, after all, UART is **A**synchronous. So we'll have to be more intentional about when we choose to read each bit we receive.

UART Receiver Sampling

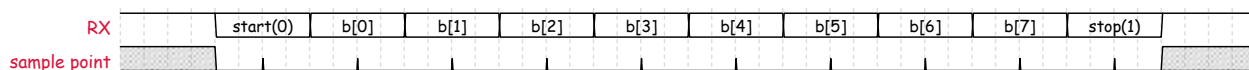


Figure 1 - When we want to sample each bit transmitted over UART. Remember, each bit is transmitted for many clock cycles, and we want to sample the bit in the middle of the period we expect it to be transmitted in.

Based on the BAUD rate, we know how long we expect each bit to appear for, but we want to make sure that slight inconsistencies in timing between the transmitter device and receiver device make as little difference as possible. If we measured each bit right as we expect it to first appear, we could unintentionally re-read the previous bit and corrupt our read data—so, instead we want to read each bit received in the **middle** of the period in which it is meant to be transmitted. This is very, very important.

Just like in our transmitter, we can calculate the `UART_BIT_PERIOD` we expect for the full period of each bit on the serial wire. Based on that value, we want to read our UART data bits `UART_BIT_PERIOD/2` clock cycles into each period.

The way we determine when our count of clock cycles begins is at the beginning of the start (0) bit. The transmission wire can sit idle (logical 1) for an unspecified amount of time, but the moment that it gets pulled low is when we need to start keeping time: the first clock cycle that reads as 0 is determined as the beginning of a data frame, and from there we count our bit periods based on that start time.

Finite State Machine: start and stop bit checking

Once we've begun reading a data frame by detecting a 0 on the receiving wire, by default we expect to see 10 bits appear afterwards, each lasting for `UART_BIT_PERIOD` cycles: a start bit of 0, the 8 bits of the transmitted data, and an end bit of 1. However, there's a chance that we're actually reading meaningless signals: maybe a loose connection or a digital glitch meant that a 0 came across the wire when it shouldn't have. That's where the start and stop bits come in handy: if the bit that's meant to be a start bit is not a 0, or if the bit that's meant to be a stop bit is not a 1, we should stop what we're doing and not interpret our signal as a valid byte.

To accomplish this, we can make a simple state machine for our UART receiver. We want our receiver to begin in an IDLE state, and get pulled out of it when we receive a 0. Then we want to progress through states to read our START bit, our DATA bits, and

Show Queue Controls

bit. But with the START and STOP bits, **our steps should be interrupted if we read incorrect bits from the wire**: when an incorrect START or STOP bit is detected, we should return to our IDLE state instead of continuing on. If both the START and STOP bits read correctly, we can progress to our TRANSMIT state and propagate our data appropriately.

To avoid glitches, after detecting a 0 on the receiving wire, you should sample again **halfway through the baud period** and make sure that the receiving wire is still a 0. This is because we want to make sure that we are actually receiving a start bit, and not just a spurious 0 caused by a digital glitch or loose wire. At this point, if this wire still 0, then we should treat the start bit as good and continue capturing the rest of the data, if not, we should go back to IDLE.

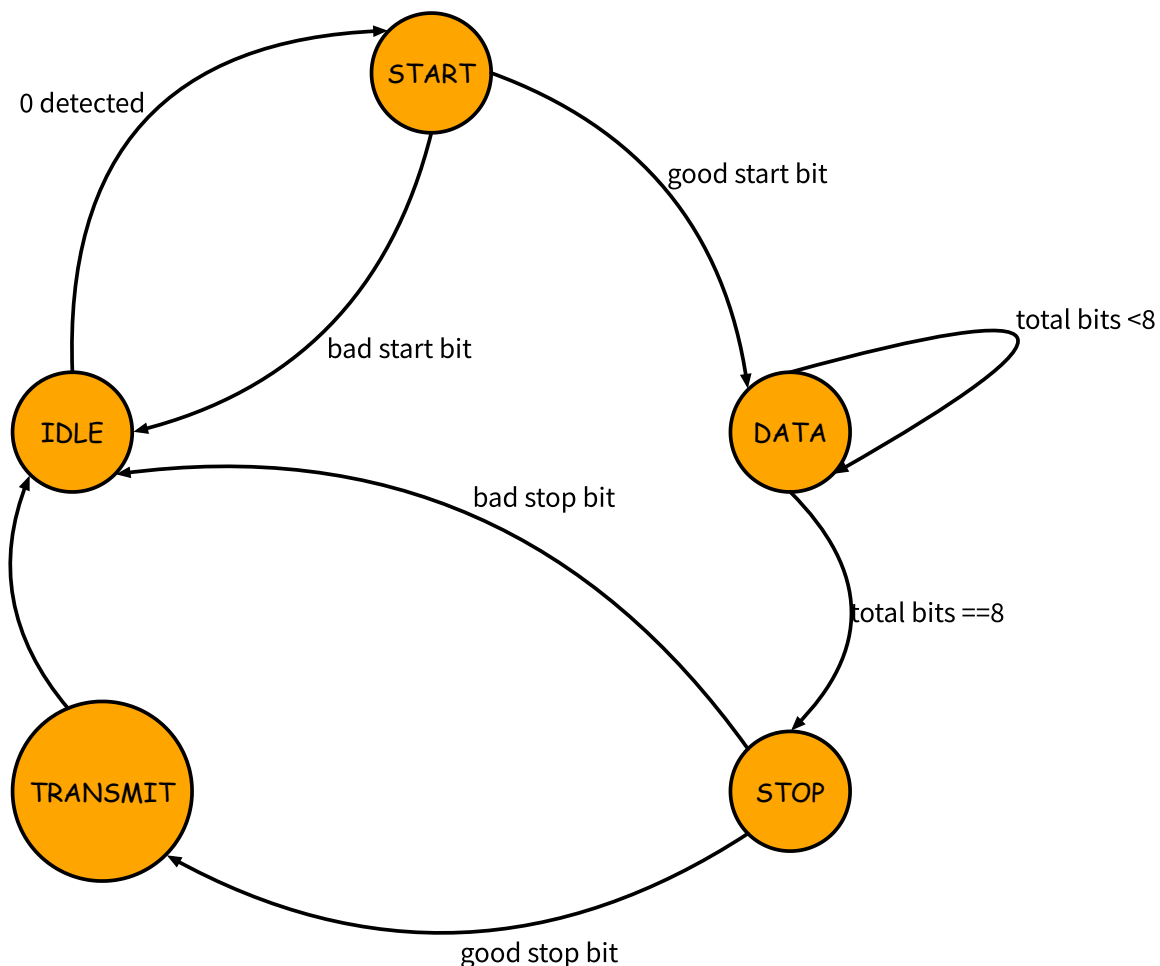


Figure 2 - UART FSM that will discard bad transmissions. Note how we can't say something is "bad" unless we're in the start or stop region. There's no way to determine if a packet is good or bad within the data region, we just have to trust the bits.

When in the START, DATA, and STOP states, your module should be counting clock cycles to measure each bit period, and counting how many bits have been received. In these states, wait for the midpoint of the bit period to sample data, and write it to a register to accumulate all the bits of the frame.

As you write your receiver module, be sure also to look back at the specification of UART from the last exercise to ensure you interpret bits in the correct order!

Fast transmitters and our state machine

In order to make a receiver that behaves reliably, especially once we're synthesizing the FPGA, we need to ensure it can handle some amount of error in terms of clock

If a transmitter connected to our receiver is operating slightly too quickly (like 1-2% too fast), we should still be able to properly capture the bytes sent. We could need this for a genuinely incorrect transmitter, or to have a buffer in case our sampling of the din wire results in slightly short or fast bit periods. The majority of our protection in this case comes from the fact that we sample in the middle of the bit period, so even if we are slightly out of sync with our transmitter we still sample the correct bit value. However, we also need to carefully consider what happens if the next start bit happens slightly before we believe the previous stop bit to end.

In order to properly handle this situation of a fast transmitter, it's important that **as soon as we check the validity of our stop bit** (i.e. halfway through the transmission of the stop bit) we should leave our STOP state; either to the TRANSMIT state if the stop bit was valid, or back to the IDLE state if the stop bit was invalid. This way, if the next start bit begins slightly early, we've already made it to the IDLE state and are ready to handle it. Additionally, the condition to move from IDLE to START should be the presence of **any** 0 on the din wire, *not a falling edge*, in case the next start bit starts earlier than expected such that there isn't enough time to measure.

Here is a code skeleton for the UART Receiver module you can use to get going.

Show/Hide Line Numbers

```

1  `timescale 1ns / 1ps
2  `default_nettype none
3
4  module uart_receive
5      #(
6          parameter INPUT_CLOCK_FREQ = 100_000_000,
7          parameter BAUD_RATE = 9600
8      )
9      (
10         input wire      clk,
11         input wire      rst,
12         input wire      din,
13         output logic     dout_valid,
14         output logic [7:0] dout
15     );
16
17     typedef enum {
18         IDLE = 0
19         // TODO: define the rest of the states your receiver needs to operate
20     } uart_state;
21
22     // note: for the online checker, don't rename this variable
23     uart_state state;
24
25     // TODO: module to read UART rx wire
26
27 endmodule // uart_receive
28
29 `default_nettype wire

```

Parameters

- parameter INPUT_CLOCK_FREQ: frequency, in Hz, of clk
- parameter BAUD_RATE: bitrate for UART transmission, in bits/second

Inputs and Outputs

- input wire clk: operational clock
- input wire rst: signal to reset the module (active high)
- input wire din: serial input signal, as described by UART



- output logic dout_valid: single-cycle high as soon as we've verified that a byte was successfully received. No need to wait for the end of the full stop bit, so long as the message was valid.
- output logic [7:0] dout: the byte received over the RX wire, valid when dout_valid is high

Internal Operation

You must design this module as a Finite State Machine, using a state variable called state with values of IDLE,START,DATA,STOP,TRANSMIT as described in the description above. The checker below will look for that to a certain extent, but it will also be verified in your checkoff on the next page (and you will not get the checkoff unless it is written as an FSM.)

module uart_receive

Submit your tested UART receiver module here.

Code Skeleton

Download Most Recent Submission

Select File

No file selected

Submit

100.00%

You have 9 submissions remaining.

Simulation Tests Completed

Test Summary

4 of 4 tests passed:

- test rx_test_single: PASSED
- test rx_test_stream: PASSED
- test rx_test_glitch: PASSED
- test fast_transmitter_test: PASSED

build log

No logged results for build.

test log

```

-.-ns INFO      gpi
-.-ns INFO      gpi
0.00ns INFO      cocotb
0.00ns INFO      cocotb
0.00ns INFO      cocotb
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.regression
0.00ns INFO      cocotb.uart_receive
130.00ns INFO     cocotb.uart_receive
9640.00ns INFO    cocotb.uart_receive
9690.00ns INFO    cocotb.regression
9690.00ns INFO    cocotb.regression
9770.00ns INFO    cocotb.uart_receive
19290.00ns INFO   cocotb.uart_receive
19770.00ns INFO   cocotb.uart_receive
19770.00ns INFO   cocotb.uart_receive
29280.00ns INFO   cocotb.uart_receive
29770.00ns INFO   cocotb.uart_receive
29770.00ns INFO   cocotb.uart_receive

..mbed/gpi_embed.cpp:108 in set_progra
../gpi/GpiCommon.cpp:101 in gpi_print_
Running on Icarus Verilog version 12.0
Running tests with cocotb v1.9.0 from /
Seeding Python random module with suppl
Found test test_rx.rx_test_single
Found test test_rx.rx_test_stream
Found test test_rx.rx_test_glitch
Found test test_rx.fast_transmitter_tes
running rx_test_single (1/4)
BAUD rate for testing: 1000000
Transmitting value 0xAB on rx_wire, ful
Module declared it has received value 0
rx_test_single passed
running rx_test_stream (2/4)
Transmitting value 0x94 on rx_wire, ful
Module declared it has received value 0
Transmission of 0x94 complete.
Transmitting value 0xEE on rx_wire, ful
Module declared it has received value 0
Transmission of 0xEE complete
Transmitting value 0x57 on r

```

39280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
39770.00ns	INFO	cocotb.uart_receive	Transmission of 0x57 complete.
39770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x39 on rx_wire, full
49280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
49770.00ns	INFO	cocotb.uart_receive	Transmission of 0x39 complete.
49770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x54 on rx_wire, full
59280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
59770.00ns	INFO	cocotb.uart_receive	Transmission of 0x54 complete.
59770.00ns	INFO	cocotb.uart_receive	Transmitting value 0xF2 on rx_wire, full
69280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
69770.00ns	INFO	cocotb.uart_receive	Transmission of 0xF2 complete.
69770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x41 on rx_wire, full
79280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
79770.00ns	INFO	cocotb.uart_receive	Transmission of 0x41 complete.
79770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x25 on rx_wire, full
89280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
89770.00ns	INFO	cocotb.uart_receive	Transmission of 0x25 complete.
89770.00ns	INFO	cocotb.uart_receive	Transmitting value 0xFF on rx_wire, full
99280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
99770.00ns	INFO	cocotb.uart_receive	Transmission of 0xFF complete.
99770.00ns	INFO	cocotb.uart_receive	Transmitting value 0xFF on rx_wire, full
109280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
109770.00ns	INFO	cocotb.uart_receive	Transmission of 0xFF complete.
109770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x0 on rx_wire, full
119280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
119770.00ns	INFO	cocotb.uart_receive	Transmission of 0x0 complete.
119770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x0 on rx_wire, full
129280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
129770.00ns	INFO	cocotb.uart_receive	Transmission of 0x0 complete.
129770.00ns	INFO	cocotb.uart_receive	Transmitting value 0xF5 on rx_wire, full
139280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
139770.00ns	INFO	cocotb.uart_receive	Transmission of 0xF5 complete.
139770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x5C on rx_wire, full
149280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
149770.00ns	INFO	cocotb.uart_receive	Transmission of 0x5C complete.
149770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x1E on rx_wire, full
159280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
159770.00ns	INFO	cocotb.uart_receive	Transmission of 0x1E complete.
159770.00ns	INFO	cocotb.uart_receive	Transmitting value 0x78 on rx_wire, full
169280.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
169770.00ns	INFO	cocotb.uart_receive	Transmission of 0x78 complete.
170770.00ns	INFO	cocotb.uart_receive	receiver detected: ['0x94', '0xee', '0x
170770.00ns	INFO	cocotb.uart_receive	transmitter sent: ['0x94', '0xee', '0x5
170770.00ns	INFO	cocotb.regression	rx_test_stream passed
170770.00ns	INFO	cocotb.regression	running rx_test_glitch (3/4)
170850.00ns	INFO	cocotb.uart_receive	Transmitting value 0xFF on rx_wire, full
180370.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
180850.00ns	INFO	cocotb.uart_receive	Transmission of 0xFF complete.
180850.00ns	INFO	cocotb.uart_receive	Transmitting value 0x80 on rx_wire, full
190360.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
190850.00ns	INFO	cocotb.uart_receive	Transmission of 0x80 complete.
190850.00ns	INFO	cocotb.uart_receive	Transmitting message 0x33 with incorrec
190850.00ns	INFO	cocotb.uart_receive	Transmitting value 0xFF on rx_wire, full
199850.00ns	INFO	cocotb.uart_receive	Transmission of 0xFF complete.
210550.00ns	INFO	cocotb.uart_receive	Transmitting brief erroneous low signal
230800.00ns	INFO	cocotb.uart_receive	Transmitting value 0xDE on rx_wire, full
240310.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
240800.00ns	INFO	cocotb.uart_receive	Transmission of 0xDE complete.
240800.00ns	INFO	cocotb.uart_receive	Transmitting value 0x0 on rx_wire, full
250310.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
250800.00ns	INFO	cocotb.uart_receive	Transmission of 0x0 complete.
250800.00ns	INFO	cocotb.uart_receive	Transmitting value 0x21 on rx_wire, full
260310.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
260800.00ns	INFO	cocotb.uart_receive	Transmission of 0x21 complete
260800.00ns	INFO	cocotb.uart_receive	Transmitting value 0x17 on r

270310.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
270800.00ns	INFO	cocotb.uart_receive	Transmission of 0x17 complete.
270830.00ns	INFO	cocotb.uart_receive	Transmitting value 0x81 on rx_wire, full
280340.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
280830.00ns	INFO	cocotb.uart_receive	Transmission of 0x81 complete.
280830.00ns	INFO	cocotb.uart_receive	Transmitting value 0x77 on rx_wire, full
290340.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
290830.00ns	INFO	cocotb.uart_receive	Transmission of 0x77 complete.
291030.00ns	INFO	cocotb.regression	rx_test_glitch passed
291030.00ns	INFO	cocotb.regression	running fast_transmitter_test (4/4)
291030.00ns	INFO	cocotb.uart_receive	Transmitter operating at 1020000, receive
291110.00ns	INFO	cocotb.uart_receive	Transmitting value 0x17 on rx_wire, full
300630.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
300910.00ns	INFO	cocotb.uart_receive	Transmission of 0x17 complete.
300910.00ns	INFO	cocotb.uart_receive	Transmitting value 0x90 on rx_wire, full
310420.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
310710.00ns	INFO	cocotb.uart_receive	Transmission of 0x90 complete.
310710.00ns	INFO	cocotb.uart_receive	Transmitting value 0xD2 on rx_wire, full
320220.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
320510.00ns	INFO	cocotb.uart_receive	Transmission of 0xD2 complete.
320510.00ns	INFO	cocotb.uart_receive	Transmitting value 0x10 on rx_wire, full
330020.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
330310.00ns	INFO	cocotb.uart_receive	Transmission of 0x10 complete.
330310.00ns	INFO	cocotb.uart_receive	Transmitting value 0xEE on rx_wire, full
339820.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
340110.00ns	INFO	cocotb.uart_receive	Transmission of 0xEE complete.
340110.00ns	INFO	cocotb.uart_receive	Transmitting value 0x90 on rx_wire, full
349620.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
349910.00ns	INFO	cocotb.uart_receive	Transmission of 0x90 complete.
349910.00ns	INFO	cocotb.uart_receive	Transmitting value 0x63 on rx_wire, full
359420.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
359710.00ns	INFO	cocotb.uart_receive	Transmission of 0x63 complete.
359710.00ns	INFO	cocotb.uart_receive	Transmitting value 0xC8 on rx_wire, full
369220.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
369510.00ns	INFO	cocotb.uart_receive	Transmission of 0xC8 complete.
369510.00ns	INFO	cocotb.uart_receive	Transmitting value 0xC4 on rx_wire, full
379020.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
379310.00ns	INFO	cocotb.uart_receive	Transmission of 0xC4 complete.
379310.00ns	INFO	cocotb.uart_receive	Transmitting value 0xE1 on rx_wire, full
388820.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
389110.00ns	INFO	cocotb.uart_receive	Transmission of 0xE1 complete.
389110.00ns	INFO	cocotb.uart_receive	Transmitting value 0x25 on rx_wire, full
398620.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
398910.00ns	INFO	cocotb.uart_receive	Transmission of 0x25 complete.
398910.00ns	INFO	cocotb.uart_receive	Transmitting value 0xFE on rx_wire, full
408420.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
408710.00ns	INFO	cocotb.uart_receive	Transmission of 0xFE complete.
408710.00ns	INFO	cocotb.uart_receive	Transmitting value 0xB7 on rx_wire, full
418220.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
418510.00ns	INFO	cocotb.uart_receive	Transmission of 0xB7 complete.
418510.00ns	INFO	cocotb.uart_receive	Transmitting value 0x6B on rx_wire, full
428020.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
428310.00ns	INFO	cocotb.uart_receive	Transmission of 0x6B complete.
428310.00ns	INFO	cocotb.uart_receive	Transmitting value 0x56 on rx_wire, full
437820.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
438110.00ns	INFO	cocotb.uart_receive	Transmission of 0x56 complete.
438110.00ns	INFO	cocotb.uart_receive	Transmitting value 0x31 on rx_wire, full
447620.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
447910.00ns	INFO	cocotb.uart_receive	Transmission of 0x31 complete.
447910.00ns	INFO	cocotb.uart_receive	Transmitting value 0x55 on rx_wire, full
457420.00ns	INFO	cocotb.uart_receive	Module declared it has received value 0
457710.00ns	INFO	cocotb.uart_receive	Transmission of 0x55 complete.
459710.00ns	INFO	cocotb.uart_receive	receiver detected: ['0x17', '0x90', '0x
459710.00ns	INFO	cocotb.uart_receive	transmitter sent: ['0x17', '0x90', '0x
459710.00ns	INFO	cocotb.regression	fast_transmitter_test passed

459710.00ns INFO cocotb.regression

```

*****
** TEST                                STATU
*****
** test_rx.rx_test_single             PASS
** test_rx.rx_test_stream             PASS
** test_rx.rx_test_glitch             PASS
** test_rx.fast_transmitter_test      PASS
*****
** TESTS=4 PASS=4 FAIL=0 SKIP=0
*****

```

Did you use a shift buffer? A UART receiver that uses variable indexing may be able to pass this checker, but will not get you a checkoff! Make sure you are using a shift buffer!

[Home](#) > [Assignments](#) > [Week 03: UART, etc...](#) > [UART Receiver](#)



Powered by [CAT-SOOP v19.0.6+git.3.26e8b68c](#) "Korat" (LTS) (development snapshot).

CAT-SOOP is [Free/Libre Software](#), available under the terms
of the [GNU Affero General Public License](#), version 3.

[Download Source Code](#)

[Javascript License Information](#)